TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

CƠ SỞ TRÍ TUỆ NHÂN TẠO

FUTOSHIKI

Giải bài toán bằng Logic vị từ bậc nhất và các thuật toán suy diễn

Giảng viên hướng dẫn: Thầy Lê Ngọc Thành
Thầy Huỳnh Lâm Hải Đăng
Thầy Lê Nhựt Nam

Sinh viên thực hiện:

Họ và tên	Mã số sinh viên	Lớp
Nguyễn Nhựt Huy	24127398	24C11
Trần Quang Minh Huy	24127403	24C11
Trịnh Khánh Linh	24127437	24C11
Nguyễn Minh Khôi	24127066	24C11

TP. Hồ Chí Minh, tháng 2/2026

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

THÔNG TIN VỀ ĐỒ ÁN

Mã học phần: CSC14003

Tên học phần: Cơ sở trí tuệ nhân tạo

Tên đồ án: Futoshiki (Giải bài toán bằng Logic vị từ bậc nhất và các thuật toán suy diễn)

Hình thức: Bài tập nhóm

Giảng viên: thầy Lê Ngọc Thành, thầy Huỳnh Lâm Hải Đăng, thầy Lê Nhựt Nam

Sinh viên thực hiện:

Nguyễn Nhựt Huy	24127398
Trần Quang Minh Huy	24127403
Trịnh Khánh Linh	24127437
Nguyễn Minh Khôi	24127066

Mục lục

Danh sách hình vẽ **1**

Danh sách bảng **2**

1 Phân công công việc **3**
1.1 Phân công chi tiết . . . 3
1.2 Yêu cầu đồ án . . . 4

2 Biểu diễn bài toán bằng Logic vị từ bậc nhất (FOL) **5**
2.1 Giới thiệu . . . 5
2.2 Từ vựng (Vocabulary) . . . 5
2.2.1 Miền giá trị . . . 5
2.2.2 Bảng vị từ . . . 5
2.3 Các tiên đề FOL . . . 5
2.4 Skolem hóa . . . 7
2.5 Chuyển đổi CNF . . . 7
2.5.1 A1 . . . 7
2.5.2 A3 . . . 7
2.5.3 A4 . . . 7
2.6 Kết luận . . . 8
2.7 Xây dựng cơ sở tri thức (Knowledge Base) . . . 8

3 Các thuật toán suy diễn **9**
3.1 Forward Chaining . . . 9
3.1.1 Cơ chế hoạt động . . . 9
3.1.2 Kết hợp Backtracking và heuristic MRV . . . 9
3.1.3 Ví dụ minh họa . . . 10
3.1.4 Mã giả thuật toán . . . 11
3.1.5 Nhận xét . . . 12
3.2 Backward Chaining . . . 12
3.2.1 Cơ chế hoạt động . . . 13
3.2.2 Ứng dụng trong truy vấn giá trị . . . 13
3.2.3 Ví dụ minh họa . . . 13
3.2.4 Mã giả thuật toán . . . 14
3.2.5 Nhận xét . . . 14
3.3 Thuật toán A* . . . 14

4 Phân tích heuristic (A*) **16**
4.1 Tổng quan áp dụng A* cho bài toán Futoshiki . . . 16
4.2 Heuristic 1 – Đếm vi phạm ràng buộc . . . 16

4.2.1 Ý tưởng . . . 16
4.2.2 Định nghĩa . . . 17
4.2.3 Mã giả . . . 17
4.2.4 Chứng minh admissible . . . 17
4.2.5 Ví dụ minh họa . . . 18
4.2.6 Nhận xét . . . 18
4.3 Heuristic 2 – AC-3 và miền giá trị . . . 18
4.3.1 Ý tưởng . . . 18
4.3.2 Định nghĩa . . . 18
4.3.3 Mã giả . . . 19
4.3.4 Chứng minh admissible . . . 19
4.3.5 Ví dụ minh họa . . . 20
4.3.6 Nhận xét . . . 20
4.4 So sánh và đánh giá hai heuristic . . . 21
4.4.1 Hạn chế của Heuristic1 . . . 21
4.4.2 Ưu điểm của Heuristic2 . . . 21
4.4.3 Ví dụ minh họa . . . 21
4.4.4 Kết luận . . . 21

5 Kết quả thực nghiệm **23**
5.1 Input-01 . . . 23
5.2 Input-02 . . . 23
5.3 Input-03 . . . 23
5.4 Input-04 . . . 23
5.5 Input-05 . . . 23
5.6 Input-06 . . . 23
5.7 Input-07 . . . 25
5.8 Input-08 . . . 26
5.9 Input-09 . . . 28

6 So sánh và đánh giá **30**

Danh sách hình vẽ

Danh sách bảng

1.1 Phân công công việc các thành viên 3
1.2 Bảng tự đánh giá yêu cầu bắt buộc 4

5.1 Kết quả benchmark cho input_06.txt 24
5.2 Kết quả benchmark cho input_07.txt 25
5.3 Kết quả benchmark cho input_08.txt 27
5.4 Kết quả benchmark cho input_09.txt 28

Phần 1
Phân công công việc

1.1 Phân công chi tiết

Bảng 1.1: Phân công công việc các thành viên

Thành viên	**Công việc được giao**	**Hoàn thành**
Nguyễn Nhựt Huy	• Quản lý repository nhóm • Cài đặt ground KB • Cài đặt thuật toán Forward Chaining • Cài đặt truy vấn Backward Chaining • Viết benchmark • Viết báo cáo kết quả thực nghiệm • Thiết kế, cài đặt GUI và visualize quá trình chạy các thuật toán	100%
Trần Quang Minh Huy	• Cài đặt thuật toán Brute-force + Backtracking • Edit video demo • Viết benchmark • Hỗ trợ các công việc chung của nhóm như chuẩn bị dữ liệu, kiểm tra kết quả.	100%
Trịnh Khánh Linh	• Biểu diễn FOL + Skolem + CNF • Viết báo cáo Forward Chaining, Backward Chaining, kết quả thực nghiệm • Quay video demo	100%
Nguyễn Minh Khôi	• Cài đặt thuật toán A* + heuristic • Viết báo cáo heuristic của A*	100%

1.2 Yêu cầu đồ án

Bảng 1.2: Bảng tự đánh giá yêu cầu bắt buộc

STT	Yêu cầu	Trạng thái	Ghi chú
1	Hình thức hóa FOL đầy đủ		Dạng Skolem & CNF cho $\geq$ 3 tiên đề
2	Sinh ground KB tự động theo N		Forward chaining, backward chaining, A*
3	Implement Forward Chaining		Viết từ đầu, đúng trên mọi test case
4	Implement Backward Chaining		Kiểu Prolog, truy vấn Val(i,j,?)
5	Implement A* Search		Heuristic chấp nhận được, có phân tích & so sánh
6	Implement Brute-force & Backtracking		Dùng để so sánh hiệu năng
7	Thiết kế $\geq$ 10 file input		Đa dạng kích thước: 4×4, 5×5, 6×6, 7×7, 9×9
8	Báo cáo đầy đủ		FOL, pseudocode, thực nghiệm (bảng + biểu đồ), so sánh thuật toán
9	Video demo		$\geq$ 3 test case, upload YouTube/Drive
10	Nộp đúng cấu trúc .zip		Docs/, Source/, README.txt, requirements.txt

Phần 2
Biểu diễn bài toán bằng Logic vị từ bậc nhất (FOL)

2.1 Giới thiệu

Futoshiki là một trò chơi giải đố logic được thực hiện trên một bảng kích thước $N \times N$. Nhiệm vụ của người chơi là điền các số từ 1 đến N vào các ô trống sao cho tất cả các ràng buộc của bài toán đều được thỏa mãn. Nhờ cấu trúc rõ ràng và các luật chơi mang tính logic chặt chẽ, Futoshiki thường được sử dụng như một ví dụ điển hình để áp dụng Lôgic Vị từ Bậc Nhất (FOL), vì các ràng buộc này có thể biểu diễn khá trực quan thông qua các mệnh đề có lượng từ.

Trong phần này, báo cáo sẽ trình bày cách xây dựng hệ tiên đề FOL cho bài toán Futoshiki kích thước 4×4. Nội dung bao gồm việc xác định từ vựng sử dụng, xây dựng các tiên đề tương ứng với luật chơi, đồng thời thực hiện Skolem hóa và chuyển đổi một số tiên đề tiêu biểu sang dạng chuẩn CNF.

2.2 Từ vựng (Vocabulary)

2.2.1 Miền giá trị

- Chỉ số hàng, cột: $i, j \in \{1, 2, 3, 4\}$
- Giá trị ô: $v \in \{1, 2, 3, 4\}$

2.2.2 Bảng vị từ

Vị từ	Bậc	Ý nghĩa
$Val(i, j, v)$	3	Ô (i, j) được gán giá trị v
$Given(i, j, v)$	3	Ô (i, j) có giá trị cho sẵn v
$LessH(i, j)$	2	Ràng buộc $<$ ngang giữa (i, j) và $(i, j+1)$
$GreaterH(i, j)$	2	Ràng buộc $>$ ngang giữa (i, j) và $(i, j+1)$
$LessV(i, j)$	2	Ràng buộc $<$ dọc giữa (i, j) và $(i+1, j)$
$GreaterV(i, j)$	2	Ràng buộc $>$ dọc giữa (i, j) và $(i+1, j)$
$Less(v1, v2)$	2	Quan hệ số học $v_1 < v_2$

2.3 Các tiên đề FOL

A1 - Mỗi ô có ít nhất một giá trị

Luật gốc: Mỗi ô trong bảng đều phải được điền một giá trị từ 1 → 4.

$$\forall i \forall j \exists v \; Val(i, j, v)$$

A2 - Mỗi ô có nhiều nhất một giá trị

Luật gốc: Mỗi ô không thể chứa đồng thời hai giá trị.

$$\forall i \forall j \forall v_1 \forall v_2 \ (Val(i,j,v_1) \wedge Val(i,j,v_2)) \Rightarrow v_1 = v_2$$

A3 - Không trùng giá trị trong hàng

Luật gốc: Mỗi hàng là một hoán vị của $\{1,2,3,4\}$, tức là không có giá trị nào trong hàng trùng nhau.

$$\forall i \forall j_1 \forall j_2 \forall v \ (Val(i,j_1,v) \wedge Val(i,j_2,v) \wedge j_1 \neq j_2) \Rightarrow \perp$$

A3′ - Không trùng giá trị trong cột

Luật gốc: Mỗi cột là một hoán vị của $\{1,2,3,4\}$, tức là không có giá trị nào trong cột trùng nhau.

$$\forall j \forall i_1 \forall i_2 \forall v \ (Val(i_1,j,v) \wedge Val(i_2,j,v) \wedge i_1 \neq i_2) \Rightarrow \perp$$

A4 - Bất đẳng thức ngang $<$

Luật gốc: Nếu có dấu "$<$"giữa hai ô (i,j) và $(i,j+1)$ thì giá trị ô trái nhỏ hơn ô phải.

$$\forall i \forall j \forall v_1 \forall v_2 \ (LessH(i,j) \wedge Val(i,j,v_1) \wedge Val(i,j+1,v_2)) \Rightarrow Less(v_1,v_2)$$

A4′ - Bất đẳng thức ngang $>$

Luật gốc: Nếu có dấu "$>$"giữa hai ô (i,j) và $(i,j+1)$ thì giá trị ô phải nhỏ hơn ô trái.

$$\forall i \forall j \forall v_1 \forall v_2 \ (GreaterH(i,j) \wedge Val(i,j,v_1) \wedge Val(i,j+1,v_2)) \Rightarrow Less(v_2,v_1)$$

A5 - Bất đẳng thức dọc $<$

Luật gốc: Nếu có dấu "$<$"giữa hai ô (i,j) và $(i+1,j)$ thì giá trị ô trên nhỏ hơn ô dưới.

$$\forall i \forall j \forall v_1 \forall v_2 \ (LessV(i,j) \wedge Val(i,j,v_1) \wedge Val(i+1,j,v_2)) \Rightarrow Less(v_1,v_2)$$

A5′ - Bất đẳng thức dọc $>$

Luật gốc: Nếu có dấu "$>$"giữa hai ô (i,j) và $(i+1,j)$ thì giá trị ô dưới nhỏ hơn ô trên.

$$\forall i \forall j \forall v_1 \forall v_2 \ (GreaterV(i,j) \wedge Val(i,j,v_1) \wedge Val(i+1,j,v_2)) \Rightarrow Less(v_2,v_1)$$

A6 - Ô cho sẵn

Luật gốc: Các ô được điền sẵn phải giữ nguyên giá trị đó.

$$\forall i \forall j \forall v \ Given(i,j,v) \Rightarrow Val(i,j,v)$$

A7 - Ràng buộc miền giá trị

Luật gốc: Chỉ được điền các số thuộc $\{1, 2, 3, 4\}$*, không được điền số ngoài miền giá trị*

$$\forall i \forall j \forall v \; Val(i, j, v) \Rightarrow (v = 1 \vee v = 2 \vee v = 3 \vee v = 4)$$

2.4 Skolem hóa

Chỉ có A1 chứa lượng từ tồn tại:

$$\forall i \forall j \exists v \; Val(i, j, v)$$

Phân tích cấu trúc lượng từ:

- $\exists v$ nằm trong phạm vi của $\forall i$ và $\forall j$.
- Vì vậy ta thay v bằng hàm Skolem $f(i, j)$ - hàm hai biến phụ thuộc vào i và j.
- Loại bỏ $\exists v$, giữ nguyên $\forall i \forall j$

Sau Skolem hóa:

$$\forall i \forall j \; Val(i, j, f(i, j))$$

$\Rightarrow$ Với mỗi cặp tọa độ (i, j) cụ thể, hàm $f(i, j)$ trả về một giá trị xác định được gán cho ô đó. Tức là, ta không cần biết cụ thể giá trị là gì - chỉ cần biết nó tồn tại và duy nhất ứng với mỗi $\forall i \forall j$.

2.5 Chuyển đổi CNF

2.5.1 A1

$$\forall i \forall j \; Val(i, j, f(i, j))$$

$$Val(i, j, f(i, j))$$

2.5.2 A3

$$\forall i \forall j_1 \forall j_2 \forall v \; (Val(i, j_1, v) \wedge Val(i, j_2, v) \wedge j_1 \neq j_2) \Rightarrow \perp$$

$$\forall i \forall j_1 \forall j_2 \forall v \; \neg[(Val(i, j_1, v) \wedge Val(i, j_2, v) \wedge j_1 \neq j_2)]$$

$$\neg Val(i, j_1, v) \vee \neg Val(i, j_2, v) \vee (j_1 = j_2)$$

2.5.3 A4

$$\forall i \forall j \forall v_1 \forall v_2 \; (LessH(i, j) \wedge Val(i, j, v_1) \wedge Val(i, j + 1, v_2)) \Rightarrow Less(v_1, v_2)$$

$$\forall i \forall j \forall v_1 \forall v_2 \; \neg[LessH(i, j) \wedge Val(i, j, v_1) \wedge Val(i, j + 1, v_2)] \vee Less(v_1, v_2)$$

$$\neg LessH(i, j) \vee \neg Val(i, j, v_1) \vee \neg Val(i, j + 1, v_2) \vee Less(v_1, v_2)$$

2.6 Kết luận

Hệ tiên đề FOL gồm 10 tiên đề mô hình hóa đầy đủ luật chơi Futoshiki. Chỉ có một tiên đề cần Skolem hóa (A1), các tiên đề còn lại thuần phổ dụng. Sau khi chuyển sang CNF, hệ có thể sử dụng cho SAT solver hoặc suy diễn logic.

2.7 Xây dựng cơ sở tri thức (Knowledge Base)

Cơ sở tri thức (KB) trong bài toán gồm hai thành phần chính:

- **Facts (Sự thật):** Là các thông tin đúng ngay từ đầu, ví dụ như các ô đã cho sẵn giá trị hoặc các ràng buộc bất đẳng thức trên bảng.
- **Rules (Luật):** Có dạng:

$$p_1 \wedge p_2 \wedge \cdots \wedge p_n \Rightarrow q$$

  trong đó các p_i là điều kiện và q là kết luận.

Ngoài ra, mỗi luật được gắn thêm một biến đếm `count` để theo dõi số lượng điều kiện chưa được thỏa mãn, giúp tối ưu quá trình suy diễn.

Từ dữ liệu đầu vào của bảng $N \times N$, nhóm tự động sinh ra KB dựa trên các loại luật sau:

- **Dữ kiện ban đầu:** Thêm các fact như `Given(i,j,v)` cho các ô đã có sẵn giá trị, và các ràng buộc như `LessH`, `GreaterV`, ...
- **Luật gán giá trị:**

$$Given(i, j, v) \Rightarrow Val(i, j, v)$$

- **Luật ràng buộc hàng và cột:** Nếu một ô đã nhận giá trị v thì các ô khác cùng hàng/cột không được nhận giá trị đó.
- **Luật bất đẳng thức:** Dựa vào các dấu $<, >$ trên bảng để loại trừ các giá trị không hợp lệ của các ô liên quan.

Phần 3
Các thuật toán suy diễn

3.1 Forward Chaining

3.1.1 Cơ chế hoạt động

Thuật toán được cài đặt theo hướng sử dụng **hàng đợi (agenda)** để quản lý các fact cần xử lý. Đồng thời, nhóm sử dụng thêm một cấu trúc chỉ mục (tương tự hash map) để nhanh chóng xác định các luật bị ảnh hưởng khi một fact mới xuất hiện.

Ý tưởng chính của Forward Chaining là lặp lại quá trình suy diễn cho đến khi đạt trạng thái ổn định (không sinh thêm fact mới). Cụ thể:

- Khởi tạo agenda với các fact ban đầu.
- Lặp lại:
  - Lấy một fact từ agenda.
  - Cập nhật các luật liên quan (giảm `count`).
  - Nếu một luật thỏa mãn (`count` = 0), suy ra fact mới.
  - Thêm fact mới vào KB và đưa vào agenda.
- Dừng khi agenda rỗng (đạt điểm cố định).

Sau bước này:

- Nếu tất cả các ô đã được xác định giá trị $\rightarrow$ kết thúc.
- Ngược lại $\rightarrow$ chuyển sang Backtracking để tiếp tục tìm nghiệm.

3.1.2 Kết hợp Backtracking và heuristic MRV

Khi Forward Chaining không thể suy diễn thêm (đạt trạng thái ổn định) nhưng bài toán vẫn chưa giải xong, nhóm sử dụng **Backtracking** để tìm kiếm phần còn lại của nghiệm.

Để giảm không gian tìm kiếm, thuật toán sử dụng heuristic **MRV (Minimum Remaining Values)**. Thay vì chọn ô theo thứ tự cố định, MRV ưu tiên chọn ô có domain nhỏ nhất (tức là ít giá trị khả thi nhất).

Ý tưởng của MRV:

- Ô có domain nhỏ thường bị ràng buộc nhiều hơn.
- Dễ phát hiện mâu thuẫn sớm nếu chọn sai.
- Giảm số nhánh cần duyệt trong cây tìm kiếm.

Quy trình Backtracking kết hợp MRV như sau:

- Chọn ô (i, j) chưa xác định với domain nhỏ nhất.
- Thử lần lượt từng giá trị v trong domain(i, j):
  - Gán $(i, j) = v$ và thêm fact `Val(i,j,v)` vào KB.
  - Chạy lại Forward Chaining để lan truyền ràng buộc.
  - Nếu xảy ra mâu thuẫn (domain rỗng hoặc conflict) thì quay lui và thử giá trị khác.

Nhờ kết hợp Forward Chaining và MRV, thuật toán vừa tận dụng được khả năng suy diễn để thu hẹp domain, vừa đảm bảo tìm được nghiệm đầy đủ với số bước thử ít hơn so với Backtracking thuần túy.

3.1.3 Ví dụ minh họa

Xét bài toán Futoshiki kích thước 3×3 với dữ liệu đầu vào như sau (intput1):

Grid:

$$\begin{bmatrix} 0 & 2 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Ràng buộc ngang:

- $(1,1) < (1,2)$

Ràng buộc dọc:

- $(1,2) > (2,2)$
- $(2,1) > (3,1)$

Bước 1: Khởi tạo KB và Domain

- Từ dữ kiện ban đầu: `Given(1,2,2)` $\Rightarrow$ `Val(1,2,2)`
- Domain ban đầu:

$$(1,2) = \{2\}, \quad \text{các ô còn lại} = \{1,2,3\}$$

Bước 2: Forward Chaining

Từ `Val(1,2,2)`, suy ra:

- Loại bỏ 2 khỏi hàng 1: $(1,1), (1,3)$ không thể là 2
- Loại bỏ 2 khỏi cột 2: $(2,2), (3,2)$ không thể là 2

Xét ràng buộc $(1,1) < (1,2)$ với $(1,2) = 2$:

- $(1,1) < 2 \Rightarrow (1,1) = 1$

Suy ra:

`Val(1,1,1)`

Tiếp tục Forward Chaining:

- Loại 1 khỏi hàng 1 và cột 1

Suy ra:

`Val(1,3,3)`

Sau một số bước suy diễn, domain được thu hẹp nhưng vẫn tồn tại trường hợp chưa xác định hết các ô.

Bước 3: Áp dụng MRV

Xét các ô chưa xác định, chọn ô có domain nhỏ nhất. Ví dụ:

- Giả sử ô $(2,2)$ còn domain $\{1,3\}$ (nhỏ nhất)

Chọn ô $(2,2)$ để thử giá trị.

Bước 4: Backtracking + Forward Chaining

- Thử $(2,2) = 1$:
  - Thêm `Val(2,2,1)` vào KB
  - Chạy lại Forward Chaining

  - Nếu xảy ra mâu thuẫn → quay lui
- Nếu không hợp lệ, thử $(2, 2) = 3$:
  - Tiếp tục suy diễn và lan truyền ràng buộc

Quá trình này tiếp tục cho đến khi tất cả các ô đều được gán giá trị.

3.1.4 Mã giả thuật toán

Algorithm 1: FC Solver cho Futoshiki

```
Input: Dữ liệu bài toán data
Output: Nghiệm solution
domain ← domain_init(data) ;
kb ← build_kb(data) ;
result ← backtrack(kb, domain) ;
if result = ∅ then
    return ∅ ;
end if
for mỗi ô (i, j) do
    v ← phần tử duy nhất trong result(i, j) ;
    data.grid[i − 1][j − 1] ← v ;
end for
return data ;
```

Algorithm 2: Backtracking với FC + MRV

```
Input: kb, domain
Output: Domain hoàn chỉnh hoặc ∅
success ← propagate(kb, domain) ;
if NOT success then
    return ∅ ;
end if
Kiểm tra tất cả ô đã xác định chưa ;
if mọi ô chỉ có 1 giá trị then
    return domain ;
end if
cell ← select_mrv(domain) ;
if cell = ∅ then
    return ∅ ;
end if
(i, j) ← cell ;
values ← các giá trị trong domain(i, j) (đã sắp xếp) ;
for mỗi v ∈ values do
    new_kb ← copy(kb) ;
    new_domain ← deepcopy(domain) ;
    new_domain(i, j) ← {v} ;
    Thêm Val(i, j, v) vào new_kb ;
    result ← backtrack(new_kb, new_domain) ;
    if result ≠ ∅ then
        return result ;
    end if
end for
return ∅ ;
```

Algorithm 3: Chọn biến theo MRV

```
Input: domain
Output: Ô (i, j) có domain nhỏ nhất
best_cell ← ∅ ;
best_size ← ∞ ;
for mỗi ô cell do
    values ← domain(cell) ;
    if |values| > 1 và |values| < best_size then
        best_size ← |values| ;
        best_cell ← cell ;
    end if
end for
return best_cell ;
```

Algorithm 4: Propagation (Forward Checking)

```
Input: kb, domain
Output: True nếu thành công, False nếu mâu thuẫn
step ← 0 ;
while True do
    old_domain ← deepcopy(domain) ;
    forward_chaining(kb) ;
    if phát hiện xung đột then
        return False ;
    end if
    Cập nhật domain từ kb ;
    if có domain rỗng then
        return False ;
    end if
    if domain = old_domain và không có fact Val mới then
        break
    end if
end while
return True ;
```

3.1.5 Nhận xét

Forward Chaining giúp lan truyền ràng buộc rất nhanh và giảm đáng kể không gian tìm kiếm ban đầu. Tuy nhiên, do chỉ dựa vào suy diễn nên thuật toán có thể bị dừng sớm khi không còn luật nào áp dụng được.

Vì vậy, việc kết hợp với Backtracking là cần thiết để đảm bảo tìm được nghiệm đầy đủ cho mọi test case.

3.2 Backward Chaining

Nếu Forward Chaining là phương pháp suy diễn tiến từ các dữ kiện ban đầu để sinh ra các sự thật mới, thì **Backward Chaining** là cách suy diễn theo hướng ngược lại: bắt đầu từ một mục tiêu cần kiểm tra và truy ngược về các luật và sự thật trong cơ sở tri thức (KB).

Trong đồ án này, Backward Chaining không được dùng để giải toàn bộ bảng, mà chủ yếu đóng vai trò **hỗ trợ truy vấn và kiểm tra tính khả thi** của một giá trị tại một ô cụ thể.

3.2.1 Cơ chế hoạt động

Thuật toán được cài đặt dưới dạng hàm đệ quy `backward_chain(kb, goal, visited, memo)`. Với một mục tiêu cần chứng minh, thuật toán thực hiện:

- Nếu mục tiêu đã tồn tại trong tập `facts` → trả về `True`.
- Ngược lại, tìm các luật có kết luận trùng với mục tiêu thông qua cấu trúc **conclusion index** (giúp truy xuất nhanh thay vì duyệt toàn bộ luật).
- Với mỗi luật tìm được, lần lượt chứng minh các tiền đề của luật bằng cách gọi đệ quy.
- Nếu tất cả tiền đề đều đúng → mục tiêu đúng.

Để tối ưu, thuật toán sử dụng thêm hai kỹ thuật:

- **Loop detection (`visited`):** Tránh lặp vô hạn khi các luật tạo thành vòng.
- **Memoization (`memo`):** Lưu kết quả các mục tiêu đã chứng minh để tránh tính lại nhiều lần.

3.2.2 Ứng dụng trong truy vấn giá trị

Trong bài toán Futoshiki, Backward Chaining được dùng để xác định các giá trị khả thi cho một ô (i, j). Thay vì chứng minh trực tiếp $Val(i, j, v)$, nhóm sử dụng cách tiếp cận đơn giản hơn:

- Với mỗi giá trị $v \in \{1, \ldots, N\}$:
  - Kiểm tra xem có chứng minh được $NotVal(i, j, v)$ hay không.
- Nếu chứng minh được → loại v khỏi domain.
- Nếu không chứng minh được → giữ v là giá trị khả thi.

Cách làm này giúp loại bỏ các giá trị không hợp lệ mà không cần thử toàn bộ không gian tìm kiếm.

3.2.3 Ví dụ minh họa

Xét bài toán Futoshiki kích thước 3×3 với dữ liệu:

Grid:

$$\begin{bmatrix} 0 & 2 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Ràng buộc:

- $(1, 1) < (1, 2)$
- $(1, 2) > (2, 2)$
- $(2, 1) > (3, 1)$

Ta cần kiểm tra xem ô $(1, 1)$ có thể nhận giá trị 2 hay không.

Bước 1: Thiết lập truy vấn

Thay vì chứng minh trực tiếp $Val(1, 1, 2)$, ta kiểm tra:

$$\texttt{backward_chain}(kb, NotVal(1, 1, 2))$$

Bước 2: Tìm luật liên quan

Từ KB, tồn tại luật ràng buộc hàng:

$$Val(1,2,2) \Rightarrow NotVal(1,1,2)$$

Bước 3: Chứng minh mục tiêu con
Để chứng minh $NotVal(1,1,2)$, ta cần chứng minh:

$$Val(1,2,2)$$

Từ dữ kiện đề bài:

$$Given(1,2,2) \Rightarrow Val(1,2,2)$$

Vì $Given(1,2,2)$ là fact $\rightarrow$ suy ra $Val(1,2,2)$ đúng.
Bước 4: Kết luận
Suy ra:

$$NotVal(1,1,2) \text{ là True}$$

Do đó, giá trị 2 bị loại khỏi domain của ô $(1,1)$.
Kết luận cuối cùng: Ô $(1,1)$ **không thể** nhận giá trị 2.
Nhận xét:
Ví dụ trên cho thấy Backward Chaining có thể nhanh chóng loại bỏ các giá trị không hợp lệ bằng cách truy ngược từ mục tiêu. Thay vì thử gán trực tiếp, thuật toán chỉ cần kiểm tra các luật liên quan trong KB, giúp giảm đáng kể số phép thử trong quá trình giải bài toán.
Kết luận: Ô $(1,2)$ không thể nhận giá trị 1.

3.2.4 Mã giả thuật toán

3.2.5 Nhận xét

Backward Chaining phù hợp cho các truy vấn cục bộ, đặc biệt khi cần kiểm tra nhanh tính hợp lệ của một giá trị.

Trong khi đó, Forward Chaining hiệu quả hơn trong việc lan truyền ràng buộc trên toàn bộ bảng. Việc kết hợp hai phương pháp giúp hệ thống vừa giảm được không gian tìm kiếm, vừa đảm bảo tính chính xác khi suy diễn.

3.3 Thuật toán A*

A* là một thuật toán tìm kiếm có thông tin (informed search), kết hợp giữa:

- Chi phí đã đi từ trạng thái ban đầu đến trạng thái hiện tại.
- Một hàm ước lượng chi phí còn lại để đạt tới trạng thái đích.

Tại mỗi bước, A* sẽ chọn trạng thái có giá trị nhỏ nhất theo hàm:

$$f(n) = g(n) + h(n)$$

trong đó:

- $g(n)$: chi phí từ trạng thái ban đầu đến trạng thái n.

- $h(n)$: hàm heuristic ước lượng chi phí từ n đến trạng thái đích.

Trong bối cảnh bài toán Futoshiki, mỗi trạng thái tương ứng với một cấu hình của bảng (một phần hoặc toàn bộ các ô đã được gán giá trị). Quá trình tìm kiếm được thực hiện bằng cách:

- Chọn một ô chưa được gán giá trị.
- Thử gán một giá trị hợp lệ từ miền giá trị của ô đó.
- Sinh ra trạng thái mới và đưa vào hàng đợi ưu tiên.

Thuật toán sử dụng một **priority queue** (hàng đợi ưu tiên) để luôn mở rộng trạng thái có giá trị $f(n)$ nhỏ nhất trước. Đồng thời, một tập **visited** được sử dụng để tránh việc lặp lại các trạng thái đã xét.

A* đảm bảo tìm được nghiệm tối ưu nếu hàm heuristic thỏa mãn tính chất admissible. Tuy nhiên, hiệu quả của thuật toán phụ thuộc rất lớn vào chất lượng của heuristic được sử dụng.

Phần 4
Phân tích heuristic (A*)

4.1 Tổng quan áp dụng A* cho bài toán Futoshiki

Điểm quan trọng của việc áp dụng A* trong bài toán Futoshiki không nằm ở cấu trúc thuật toán, mà ở cách sử dụng heuristic để định hướng quá trình điền giá trị vào bảng.

Tại mỗi trạng thái, thuật toán thực hiện các bước sau:

- **Chọn biến cần gán:** Một ô chưa được điền sẽ được chọn để mở rộng. Việc lựa chọn này có thể dựa trên các tiêu chí như thứ tự duyệt hoặc ưu tiên các ô có nhiều ràng buộc.
- **Sinh các trạng thái con:** Với mỗi giá trị hợp lệ trong domain của ô đã chọn, tạo ra một trạng thái mới bằng cách gán giá trị đó vào ô tương ứng.
- **Áp dụng ràng buộc:** Sau khi gán, các ràng buộc được cập nhật (có thể kết hợp AC-3 hoặc forward checking) để thu hẹp domain của các ô còn lại.
- **Đánh giá heuristic:** Với mỗi trạng thái mới, giá trị $h(n)$ được tính toán dựa trên:
  - Số lượng ràng buộc chưa thỏa mãn (Heuristic1), hoặc
  - Mức độ chưa xác định của các biến thông qua domain (Heuristic2)
- **Tính giá trị hàm đánh giá:**
$$f(n) = g(n) + h(n)$$
  Trong đó:
  - $g(n)$ là số bước đã gán (số ô đã điền thêm)
  - $h(n)$ phản ánh mức độ còn lại cần hoàn thành
- **Ưu tiên mở rộng:** Trạng thái có giá trị $f(n)$ nhỏ nhất sẽ được chọn để tiếp tục mở rộng.

Nhờ việc sử dụng heuristic, A* không duyệt tất cả các khả năng như brute-force mà tập trung vào các trạng thái "hứa hẹn" hơn, tức là các trạng thái:

- Có ít vi phạm ràng buộc, hoặc
- Có domain đã được thu hẹp đáng kể

Điều này giúp giảm đáng kể số lượng trạng thái cần xét và tăng hiệu quả giải bài toán.

4.2 Heuristic 1 – Đếm vi phạm ràng buộc

4.2.1 Ý tưởng

Heuristic1 đánh giá trạng thái dựa trên số lượng ô vi phạm các ràng buộc bất đẳng thức.

Một ô được xem là vi phạm nếu tồn tại ít nhất một ràng buộc:

- Không thỏa mãn bất đẳng thức, hoặc
- Chưa xác định do liên quan đến ô có giá trị 0

4.2.2 Định nghĩa

$$h(n) = \text{số ô vi phạm}$$

4.2.3 Mã giả

Heuristic1

```
FUNCTION Heuristic1(state, data):
    count ← 0

    FOR mỗi ô (i, j):
        IF tồn tại ràng buộc vi phạm:
            count ← count + 1

    RETURN count
```

4.2.4 Chứng minh admissible

Gọi:

- V là tập các ô bị vi phạm
- $|V|$ là số lượng ô vi phạm

Ta có:

- Mỗi ô vi phạm cần được xử lý ít nhất một lần để thỏa mãn ràng buộc.
- Mỗi bước gán chỉ có thể thay đổi một ô.

Do đó, chi phí tối thiểu để đạt nghiệm thỏa mãn:

$$h^*(n) \geq |V|$$

Trong khi heuristic:

$$h(n) = |V|$$

Suy ra:

$$h(n) \leq h^*(n)$$

$$\Rightarrow \text{Heuristic1 là admissible}$$

4.2.5 Ví dụ minh họa

Ví dụ 3x3

```
Grid:
0   2   0
0   0   0
0   0   0

Constraints:
(1,1) < (1,2)
(1,2) > (2,2)
(2,1) > (3,1)
```

Các ô vi phạm: $(1,1), (1,2), (2,1), (2,2)$

$$h(n) = 4$$

4.2.6 Nhận xét

- Ưu điểm: đơn giản, admissible.
- Nhược điểm: đánh giá còn thô, chưa phản ánh sâu cấu trúc bài toán.

4.3 Heuristic 2 – AC-3 và miền giá trị

4.3.1 Ý tưởng

Heuristic2 sử dụng thuật toán AC-3 để lan truyền ràng buộc và đánh giá mức độ chưa xác định của trạng thái.

- Nếu domain rỗng $\rightarrow$ trạng thái vô nghiệm.
- Ngược lại, đếm số biến có nhiều hơn một giá trị khả dĩ.

4.3.2 Định nghĩa

$$h(n) = \begin{cases} \infty & \text{nếu có domain rỗng} \\ \text{số biến có } |domain| > 1 & \text{ngược lại} \end{cases}$$

4.3.3 Mã giả

Heuristic2

```
FUNCTION Heuristic2(state, data):
    domains ← copy(state.domains)

    IF AC3(domains) == FALSE:
        RETURN INFINITY

    count ← 0
    FOR mỗi domain d:
        IF size(d) > 1:
            count ← count + 1

    RETURN count
```

4.3.4 Chứng minh admissible

Heuristic2 được định nghĩa là số lượng biến có miền giá trị lớn hơn 1 sau khi áp dụng AC-3. Mặc dù trực quan cho thấy heuristic này phản ánh số lượng ô chưa được xác định, nhưng nó **không đảm bảo tính admissible**.

4.3.4.1 Phản ví dụ

Xét một trạng thái sau khi áp dụng AC-3:

Biến	Domain
A	{1,2}
B	{1,2}

Giả sử tồn tại ràng buộc giữa A và B sao cho khi một biến được gán giá trị, biến còn lại sẽ được xác định duy nhất.

Khi đó:

- Giá trị heuristic:

$$h(n) = 2$$

- Tuy nhiên, chỉ cần một phép gán:

$$A = 1 \Rightarrow B = 2$$

Suy ra chi phí thực tế:

$$h^*(n) = 1$$

Do đó:

$$h(n) > h^*(n) \Rightarrow \text{Heuristic2 không admissible}$$

4.3.4.2 Tính hữu ích của heuristic

Mặc dù không admissible, Heuristic2 vẫn mang lại hiệu quả cao trong thực tế nhờ các đặc điểm sau:

- **Phát hiện sớm trạng thái vô nghiệm:** Khi AC-3 trả về domain rỗng, heuristic ngay lập tức loại bỏ trạng thái đó khỏi quá trình tìm kiếm.
- **Phản ánh mức độ chưa xác định:** Số lượng biến có domain lớn hơn 1 cho thấy mức độ "chưa hoàn chỉnh" của trạng thái. Trạng thái có giá trị heuristic nhỏ thường gần nghiệm hơn.
- **Kết hợp với propagation:** Việc sử dụng AC-3 giúp lan truyền ràng buộc trước khi đánh giá heuristic, từ đó giảm đáng kể không gian tìm kiếm.
- **Định hướng tìm kiếm hiệu quả:** So với các heuristic đơn giản, Heuristic2 giúp A* ưu tiên các trạng thái có domain đã được thu hẹp, từ đó giảm số lượng trạng thái cần mở rộng.

4.3.4.3 Kết luận

Heuristic2 đánh đổi tính admissible để đạt được khả năng đánh giá trạng thái chính xác hơn trong thực tế. Mặc dù không đảm bảo tìm nghiệm tối ưu trong mọi trường hợp, heuristic này giúp cải thiện đáng kể hiệu năng tìm kiếm của thuật toán A*.

4.3.5 Ví dụ minh họa

Sau khi áp dụng AC-3 cho cùng ví dụ trên, ta có:

Ô	Domain	Size
(1,1)	{1}	1
(1,2)	{2}	1
(1,3)	{1,2,3}	3
(2,1)	{1,2,3}	3
(2,2)	{1}	1
(2,3)	{1,2,3}	3
(3,1)	{1,2,3}	3
(3,2)	{1,2,3}	3
(3,3)	{1,2,3}	3

Có 6 biến có $|domain| > 1$

$$h(n) = 6$$

4.3.6 Nhận xét

- Ưu điểm:
  - Phát hiện sớm trạng thái vô nghiệm
  - Phản ánh mức độ chưa xác định tốt hơn
- Nhược điểm:
  - Chi phí tính toán cao hơn
  - Không đảm bảo admissible

4.4 So sánh và đánh giá hai heuristic

Để đánh giá hiệu quả của hai heuristic, ta xét khả năng **định hướng tìm kiếm** của chúng trong thuật toán A*.

4.4.1 Hạn chế của Heuristic1

Heuristic1 chỉ dựa trên việc đếm số ô vi phạm ràng buộc. Tuy nhiên, cách đánh giá này tồn tại một số hạn chế:

- **Thông tin cục bộ:** Heuristic1 chỉ xem xét các ràng buộc bị vi phạm trực tiếp, mà không phản ánh được mức độ ràng buộc tổng thể của toàn bộ bảng.
- **Không phân biệt được các trạng thái tương đương:** Nhiều trạng thái có cùng số vi phạm nhưng mức độ "gần nghiệm" rất khác nhau.

4.4.2 Ưu điểm của Heuristic2

Heuristic2 sử dụng AC-3 để thu hẹp miền giá trị trước khi đánh giá trạng thái, do đó mang lại các ưu điểm sau:

- **Khai thác thông tin toàn cục:** Thông qua AC-3, các ràng buộc được lan truyền trên toàn bộ bảng, giúp heuristic phản ánh chính xác hơn trạng thái hiện tại.
- **Phân biệt tốt hơn giữa các trạng thái:** Hai trạng thái có cùng số vi phạm có thể có domain rất khác nhau, và Heuristic2 có thể nhận ra sự khác biệt này.
- **Giảm không gian tìm kiếm:** Việc loại bỏ sớm các giá trị không hợp lệ giúp giảm số lượng trạng thái cần xét trong quá trình tìm kiếm.

4.4.3 Ví dụ minh họa

Xét hai trạng thái có cùng số vi phạm theo Heuristic1:

- Trạng thái A: các domain đều lớn (nhiều giá trị khả dĩ)
- Trạng thái B: nhiều domain đã bị thu hẹp (gần xác định)

Khi đó:

- Heuristic1: $h_1(A) = h_1(B)$ (không phân biệt được)
- Heuristic2: $h_2(A) > h_2(B)$ (phân biệt rõ trạng thái tốt hơn)

Do đó, A* sử dụng Heuristic2 sẽ ưu tiên trạng thái B, giúp tiến gần nghiệm nhanh hơn.

4.4.4 Kết luận

Tiêu chí	Heuristic1	Heuristic2
Ý tưởng	Đếm vi phạm	Domain + AC-3
Admissible	Có	Không đảm bảo
Độ chính xác	Thấp	Cao hơn
Chi phí tính	Thấp	Cao

Heuristic2 cung cấp thông tin giàu hơn so với Heuristic1 nhờ việc kết hợp constraint propagation. Mặc dù không đảm bảo tính admissible, heuristic này giúp định hướng tìm kiếm hiệu quả hơn, từ đó giảm số lượng trạng thái cần mở rộng và cải thiện thời gian giải bài toán trong thực tế.

Heuristic1 phù hợp khi cần đảm bảo tính tối ưu của A*, trong khi Heuristic2 giúp cải thiện hiệu năng tìm kiếm nhờ đánh giá trạng thái chính xác hơn.

Việc lựa chọn heuristic phụ thuộc vào mục tiêu giữa tối ưu nghiệm và thời gian thực thi.

Phần 5
Kết quả thực nghiệm

5.1 Input-01

Mô tả testcase: Kết quả: So sánh và đánh giá:

5.2 Input-02

Mô tả testcase: Kết quả: So sánh và đánh giá:

5.3 Input-03

Mô tả testcase: Kết quả: So sánh và đánh giá:

5.4 Input-04

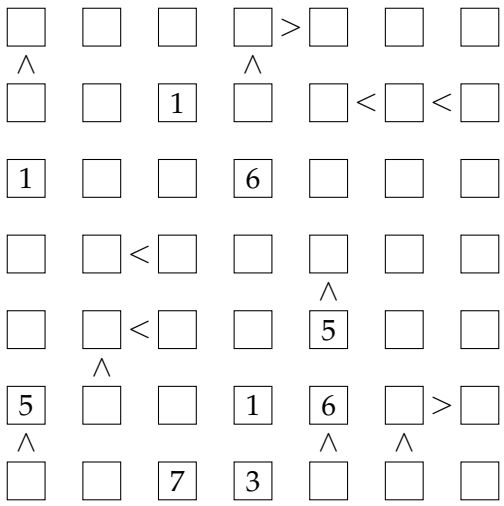
Mô tả testcase: Kết quả: So sánh và đánh giá:

5.5 Input-05

Mô tả testcase: Kết quả: So sánh và đánh giá:

5.6 Input-06

Mô tả testcase:



Kết quả:

Input	Method	Runtime (s)	Memory (MB)	Nodes	Solved	Timeout
input_06.txt	Forward Chaining	0.09	3.05	1	False	False
input_06.txt	A* (Heuristic 1)	300.00	1905.46	182,217	False	True
input_06.txt	A* (Heuristic 2)	300.00	325.93	80,879	False	True
input_06.txt	Brute Force	300.00	0.15	2,428,877	False	True
input_06.txt	Backtracking	314.86	10589.39	2,237,011	False	True

Bảng 5.1: Kết quả benchmark cho input_06.txt

So sánh và đánh giá:

- **Forward Chaining:**
  - Thời gian chạy rất thấp (0.089s), bộ nhớ sử dụng nhỏ (3.05MB), và chỉ mở rộng 1 node.
  - Tuy nhiên, không tìm được nghiệm.
  - Điều này cho thấy phương pháp suy luận tiến chỉ thực hiện propagation ràng buộc nhưng không có cơ chế tìm kiếm, do đó thất bại khi bài toán yêu cầu khám phá không gian trạng thái sâu hơn.
- **A* với Heuristic 1:**
  - Bị timeout sau 300s, mở rộng 182,217 nodes và sử dụng bộ nhớ lớn (1905.46MB).
  - Hiệu năng kém cho thấy heuristic 1 không đủ mạnh để định hướng tìm kiếm, dẫn đến việc mở rộng nhiều trạng thái không cần thiết.
- **A* với Heuristic 2:**
  - Cũng bị timeout, nhưng số node mở rộng giảm đáng kể (80,879 nodes) và bộ nhớ thấp hơn (325.93MB) so với Heuristic 1.
  - Điều này chứng tỏ heuristic 2 có khả năng định hướng tốt hơn, giúp giảm không gian tìm kiếm.
  - Tuy nhiên, vẫn chưa đủ hiệu quả để giải bài toán trong giới hạn thời gian.
- **Brute Force:**
  - Mở rộng số lượng node cực lớn (2,428,877 nodes) và bị timeout.
  - Bộ nhớ sử dụng thấp (0.15MB) do không lưu nhiều trạng thái.
  - Đây là phương pháp kém hiệu quả nhất về mặt thời gian do không có bất kỳ cơ chế cắt tỉa nào.
- **Backtracking:**
  - Tương tự brute force nhưng có cải thiện nhẹ nhờ cắt tỉa một phần.
  - Tuy nhiên vẫn mở rộng hơn 2 triệu nodes và vượt quá thời gian giới hạn (314.86s).
  - Bộ nhớ rất cao (10589.39MB), cho thấy chi phí lưu trạng thái và recursion stack lớn.

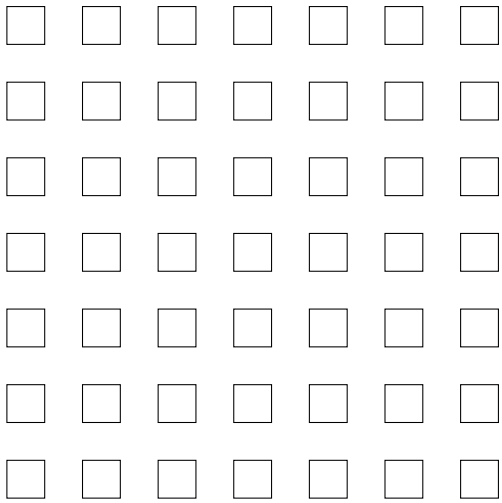
So sánh tổng thể:

- Về **khả năng giải bài**: không có phương pháp nào tìm được nghiệm trong thời gian giới hạn.
- Về **hiệu quả heuristic**:
  - Heuristic 2 tốt hơn Heuristic 1 (ít node hơn, ít bộ nhớ hơn).
  - Tuy nhiên cả hai vẫn chưa đủ mạnh để xử lý testcase khó này.
- Về **chi phí tìm kiếm**:
  - Brute Force và Backtracking có chi phí rất lớn (hàng triệu node).
  - A* giúp giảm đáng kể số node nhưng vẫn chưa đủ.

- Về **suy luận ràng buộc**:
  - Forward Chaining rất nhanh nhưng không đủ để giải bài toán khi thiếu bước tìm kiếm.

5.7 Input-07

Mô tả testcase:



Kết quả:

Input	Method	Runtime (s)	Memory (MB)	Nodes	Solved	Timeout
input_07.txt	Forward Chaining	0.62	36.87	26	True	False
input_07.txt	A* (Heuristic 1)	300.00	2768.83	73,799	False	True
input_07.txt	A* (Heuristic 2)	1.28	1.51	102	True	False
input_07.txt	Brute Force	0.01	0.01	132	True	False
input_07.txt	Backtracking	0.00	0.39	132	True	False

Bảng 5.2: Kết quả benchmark cho input_07.txt

So sánh và đánh giá:

- **Forward Chaining:**
  - Thời gian chạy thấp (0.62s), số node mở rộng rất ít (26 nodes).
  - Tìm được nghiệm thành công.
  - Mặc dù không có nhiều ràng buộc để suy luận, phương pháp vẫn hoạt động hiệu quả do bài toán không yêu cầu tìm kiếm sâu.
- **A* với Heuristic 1:**
  - Bị timeout sau 300s, mở rộng 73,799 nodes và tiêu tốn bộ nhớ lớn (2768.83MB).
  - Hiệu năng kém do heuristic không cung cấp thông tin hữu ích trong trường hợp không có ràng buộc, dẫn đến việc tìm kiếm gần như mù.
- **A* với Heuristic 2:**
  - Giải được bài toán trong thời gian ngắn (1.28s), mở rộng 102 nodes và sử dụng bộ nhớ rất thấp (1.51MB).

  - Hoạt động tốt hơn Heuristic 1 nhờ khả năng định hướng tìm kiếm hợp lý hơn, dù bài toán không có nhiều thông tin ràng buộc.

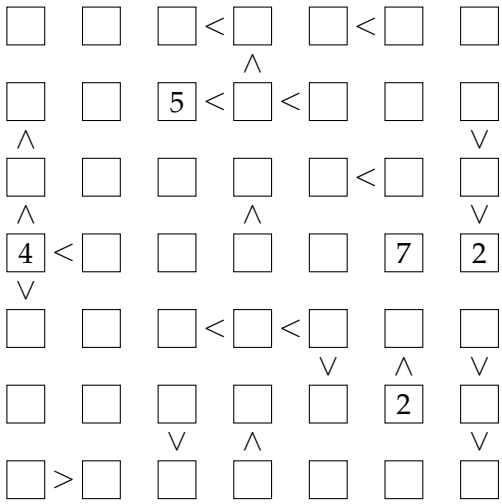
- **Brute Force:**
  - Giải rất nhanh (0.01s) với 132 nodes.
  - Do bài toán có nhiều nghiệm, brute force dễ dàng tìm được một nghiệm mà không cần khám phá toàn bộ không gian.
- **Backtracking:**
  - Là phương pháp nhanh nhất (0.0045s), với 132 nodes.
  - Hiệu quả cao nhờ dừng sớm khi tìm được nghiệm đầu tiên.

So sánh tổng thể:

- Về **khả năng giải bài**: hầu hết các phương pháp (trừ A* Heuristic 1) đều giải được nhanh chóng.
- Về **hiệu quả heuristic**:
  - Heuristic 2 vượt trội so với Heuristic 1.
  - Heuristic 1 không phù hợp trong trường hợp thiếu ràng buộc.
- Về **chi phí tìm kiếm**:
  - Brute Force và Backtracking có chi phí rất thấp do dừng sớm.
  - A* (H1) có chi phí rất cao do mở rộng nhiều node không cần thiết.
- Về **ảnh hưởng của ràng buộc**:
  - Việc không có constraint khiến bài toán trở nên "dễ" theo nghĩa tìm nghiệm đầu tiên.
  - Tuy nhiên, heuristic trở nên kém hiệu quả do thiếu thông tin định hướng.

5.8 Input-08

Mô tả testcase:



Kết quả:

Input	Method	Runtime (s)	Memory (MB)	Nodes	Solved	Timeout
input_08.txt	Forward Chaining	35.35	37.11	1,090	True	False
input_08.txt	A* (Heuristic 1)	300.00	1747.70	271,563	False	True
input_08.txt	A* (Heuristic 2)	5.71	2.60	132	True	False
input_08.txt	Brute Force	300.00	0.15	2,253,684	False	True
input_08.txt	Backtracking	5.11	479.79	101,260	True	False

Bảng 5.3: Kết quả benchmark cho input_08.txt

So sánh và đánh giá:

- **Forward Chaining:**
  - Giải được bài toán trong 35.35s với 1,090 nodes.
  - Bộ nhớ sử dụng ở mức trung bình (37.11MB).
  - Cho thấy suy luận ràng buộc có hiệu quả nhất định trong việc giảm không gian tìm kiếm, nhưng vẫn cần thời gian đáng kể để hội tụ.
- **A* với Heuristic 1:**
  - Bị timeout sau 300s, mở rộng 271,563 nodes và tiêu tốn bộ nhớ lớn (1747.7MB).
  - Heuristic 1 không đủ mạnh để định hướng tìm kiếm hiệu quả, dẫn đến việc khám phá nhiều trạng thái không cần thiết.
- **A* với Heuristic 2:**
  - Giải thành công trong thời gian ngắn (5.71s), chỉ mở rộng 132 nodes.
  - Bộ nhớ sử dụng rất thấp (2.6MB).
  - Đây là phương pháp hiệu quả nhất trong testcase này, cho thấy heuristic 2 có khả năng định hướng tìm kiếm rất tốt.
- **Brute Force:**
  - Bị timeout với số node mở rộng rất lớn (2,253,684 nodes).
  - Không có cơ chế cắt tỉa nên không phù hợp với bài toán có không gian tìm kiếm lớn.
- **Backtracking:**
  - Giải được bài toán trong 5.11s, nhưng mở rộng tới 101,260 nodes.
  - Bộ nhớ khá cao (479.79MB).
  - Mặc dù có cắt tỉa, nhưng vẫn kém hiệu quả hơn A* với heuristic tốt.

So sánh tổng thể:

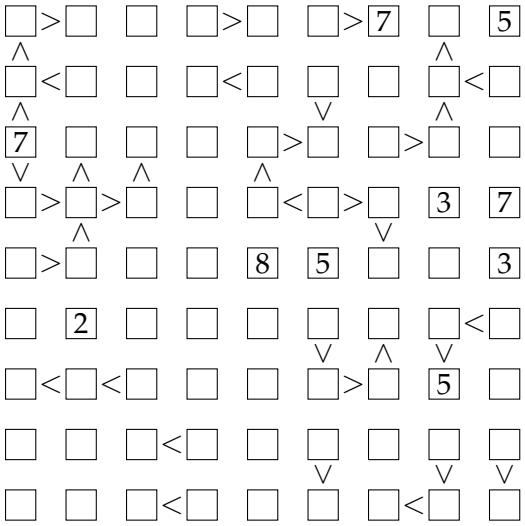
- Về **khả năng giải bài**:
  - Forward Chaining, A* (Heuristic 2) và Backtracking đều giải được bài toán.
  - A* (Heuristic 1) và Brute Force thất bại do timeout.
- Về **hiệu quả heuristic**:
  - Heuristic 2 vượt trội so với Heuristic 1 cả về thời gian, bộ nhớ và số node mở rộng.
  - Điều này cho thấy tầm quan trọng của việc thiết kế heuristic phù hợp.
- Về **chi phí tìm kiếm**:
  - A* (Heuristic 2) có chi phí thấp nhất (132 nodes).

  - Backtracking tốn nhiều node hơn nhưng vẫn trong giới hạn chấp nhận được.
  - Brute Force là kém nhất do không có cắt tỉa.

- Về **vai trò của ràng buộc**:
  - Các constraint giúp giảm đáng kể không gian tìm kiếm, cho phép các thuật toán như A* (H2) hoạt động hiệu quả.

5.9 Input-09

Mô tả testcase:



Kết quả:

Input	Method	Runtime (s)	Memory (MB)	Nodes	Solved	Timeout
input_09.txt	Forward Chaining	300.15	194.51	3,145	False	True
input_09.txt	A* (Heuristic 1)	300.00	2834.59	66,178	False	True
input_09.txt	A* (Heuristic 2)	300.01	204.55	4,138	False	True
input_09.txt	Brute Force	300.00	0.15	1,151,494	False	True
input_09.txt	Backtracking	300.72	10602.58	1,722,995	False	True

Bảng 5.4: Kết quả benchmark cho input_09.txt

So sánh và đánh giá:

- **Forward Chaining:**
  - Bị timeout sau 300s, chỉ mở rộng 3,145 nodes.
  - Bộ nhớ sử dụng ở mức trung bình (194.51MB).
  - Mặc dù số node mở rộng không lớn, phương pháp không thể hội tụ do thiếu cơ chế tìm kiếm sâu, chỉ dựa vào suy luận ràng buộc.
- **A* với Heuristic 1:**
  - Bị timeout với 66,178 nodes và bộ nhớ rất cao (2834.59MB).
  - Heuristic 1 không đủ mạnh để định hướng tìm kiếm trong không gian trạng thái lớn, dẫn đến việc mở rộng nhiều trạng thái không cần thiết.
- **A* với Heuristic 2:**

  - Cũng bị timeout, nhưng chỉ mở rộng 4,138 nodes và sử dụng bộ nhớ thấp hơn (204.55MB).
  - So với Heuristic 1, hiệu quả tốt hơn rõ rệt, cho thấy heuristic 2 có khả năng định hướng tốt hơn.
  - Tuy nhiên, vẫn chưa đủ mạnh để giải bài toán trong thời gian giới hạn.

- **Brute Force:**
  - Mở rộng hơn 1.15 triệu nodes và bị timeout.
  - Bộ nhớ thấp (0.15MB) do không lưu trữ nhiều trạng thái.
  - Đây là phương pháp kém hiệu quả nhất về thời gian do không có cơ chế cắt tỉa.
- **Backtracking:**
  - Mở rộng hơn 1.7 triệu nodes và vượt thời gian giới hạn (300.72s).
  - Bộ nhớ rất cao (10602.58MB), phản ánh chi phí lớn trong việc duy trì trạng thái và recursion.
  - Có cải thiện so với brute force nhờ cắt tỉa, nhưng vẫn không đủ hiệu quả.

So sánh tổng thể (input_09.txt và input_10.txt):

- Về **khả năng giải bài**: tất cả các phương pháp đều thất bại (timeout), cho thấy đây là testcase cực khó.
- Về **hiệu quả heuristic**:
  - Heuristic 2 tốt hơn Heuristic 1 (ít node hơn, ít bộ nhớ hơn).
  - Tuy nhiên, cả hai đều không đủ mạnh để giải bài toán.
- Về **chi phí tìm kiếm**:
  - Brute Force và Backtracking có chi phí cực lớn (hàng triệu node).
  - A* giúp giảm đáng kể số node, đặc biệt với Heuristic 2.
- Về **vai trò của ràng buộc**:
  - Mặc dù có nhiều ràng buộc, chúng vẫn chưa đủ mạnh để thu hẹp không gian tìm kiếm một cách hiệu quả.

Phần 6
So sánh và đánh giá